GUIDE 41 · EVENTS & EXPORTS

Publish Parquet, CSV or JSON files in S3 as an API

The latest version is at anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=41-publish-files-in-s3-as-an-api. This copy was made on 4 November 2026.

Time: 15 minutes. **You'll end up with:** files in a bucket - or on a web server, or a local disk - served as governed REST endpoints and MCP tools, with no database to load them into: DuckDB reads them where they are.

Every command and response below was run against an S3-compatible server (SeaweedFS) holding `s3://sales/2026/orders.parquet` - 5,000 orders - and a second bucket, `hrdata`, that the API must never reach.

Step 1: Decide what the connection may read

A DuckDB connection reads exactly the paths in its `allowed_paths` and nothing else - DuckDB itself refuses the rest, however a query asks. So start by naming the narrowest prefix that holds your files: here `s3://sales/2026/`.

Give it **credentials that can read only that prefix** too (a bucket policy or IAM role). The allow-list is enforced by DuckDB; the credentials are enforced by the storage service - two locks, and the second is the one that holds even if the first were ever bypassed.

Step 2: Create the connection

```shell
curl -X POST http://127.0.0.1:5000/api/v1/connections -H 'X-API-Key: demo-key' -H 'Content-Type:
application/json' -d '{
  "name": "lake", "db": "duckdb", "database": ":memory:", "active": true,
  "allowed_paths": ["s3://sales/2026/"],
  "user": "AKIA...", "password": "${LAKE_SECRET_KEY}", "region": "eu-west-1",
  "views": {"orders": "SELECT * FROM read_parquet('''s3://sales/2026/*.parquet''')"}
}'
```

- `database: ":memory:"` - the connection holds no tables of its own, only views over files.
- `user` and `password` are the access key ID and secret. The secret is handled like any connection password - here a `${LAKE_SECRET_KEY}` reference to the server's environment; a typed one is masked and, with `QUERYAPIGATE_SECRET_KEY`, encrypted (guide 22). Leave both out for a public bucket.
- For an S3-compatible service (MinIO, SeaweedFS, Ceph), add `"endpoint": "minio.internal:9000"` and `"url_style": "path"` (and `"use_ssl": false` without TLS). For Google Cloud Storage use `"storage": "gcs"` with HMAC keys; for Cloudflare R2, `"storage": "r2"` and `"account_id"`.
- `views` gives the files a name: `orders`. Or let the connection name them: `"auto_views": true` makes a view of each file and subfolder under each allowed prefix - `s3://sales/2026/orders.parquet` becomes `orders`, a subfolder `returns/` becomes `returns` over every file in it (the rules).

In the Console: **Connections → New connection**, type **Files (Parquet, CSV, JSON)** - no database to fill in, a **Files it may read** section, and automatic views already on.

`POST /api/v1/connections/test` with the same body tries it first: `{"elapsed_ms": 118.6}`. The server needs DuckDB's `httpfs` extension; it's in the Docker image, and elsewhere downloaded on first use.

The view shows up in the schema browser with its columns, read from the Parquet file itself:

```json
{"tables": [{"name": "orders", "type": "view", "columns": [
  {"name": "order_id", "type": "BIGINT", ...}, {"name": "region", "type": "VARCHAR", ...},
  {"name": "total", "type": "DECIMAL(22,1)", ...}, {"name": "day", "type": "DATE", ...}]}], "truncated":
false}
```

Step 3: Save a query

Exactly as for any database (guide 1):

```shell
curl -X POST http://127.0.0.1:5000/api/v1/queries -H 'X-API-Key: demo-key' -H 'Content-Type:
application/json' -d '{
  "name": "daily_revenue", "connection_name": "lake", "publish": true,
  "description": "Revenue per day in a date range",
  "sql": "SELECT day, COUNT(*) AS orders, SUM(total) AS revenue FROM orders WHERE day BETWEEN :from AND
:to GROUP BY day ORDER BY day",
  "parameters": {"from": {"type": "str", "pattern": "\\d{4}-\\d{2}-\\d{2}"},
                 "to": {"type": "str", "pattern": "\\d{4}-\\d{2}-\\d{2}"}}
}'

curl 'http://127.0.0.1:5000/q/daily_revenue?from=2026-09-01&to=2026-09-03' -H 'X-API-Key: demo-key'
```

```json
[{"day": "2026-09-01", "orders": 167, "revenue": 4158300.0},
 {"day": "2026-09-02", "orders": 167, "revenue": 4159970.0},
 {"day": "2026-09-03", "orders": 167, "revenue": 4161640.0}]
```

Everything else works too - formats, `?stream=true` exports, caching, run history, MCP tools. Each request reads the files, so for a dashboard polled often, add `cache_ttl` (guide 8): the files are read once per TTL, not once per poll.

Step 4: Hand it out

A partner or an app gets the saved query only - no `connections` grant, so no SQL of its own (guide 14):

```shell
curl -X POST http://127.0.0.1:5000/api/v1/api-keys -H 'X-API-Key: demo-key' -H 'Content-Type:
application/json' \
  -d '{"name": "finance-dashboard", "connections": [], "queries": ["daily_revenue"], "rate_limit":
"60/minute"}'
```

An analyst who should write their own SQL - over the view, not the bucket - gets the connection with `allowed_tables`:

```shell
curl -X POST http://127.0.0.1:5000/api/v1/api-keys -H 'X-API-Key: demo-key' -H 'Content-Type:
application/json' \
  -d '{"name": "analyst", "connections": ["lake"], "allowed_tables": ["orders"]}'
```

Verified, for that key:

SQL	RESULT
`SELECT region, SUM(total) AS revenue FROM orders GROUP BY region`	`[{"region": "EU", "revenue": 124975000.0}]`
`SELECT * FROM read_parquet('s3://sales/2026/orders.parquet')`	`403 table_not_allowed` - a table-restricted key can't call file-reading functions

What can't be reached

Even the admin key can't read outside `allowed_paths`:

```shell
# SELECT * FROM read_parquet('s3://hrdata/salaries.parquet')
# {"error": "This connection may not read 's3://hrdata/salaries.parquet' - it isn't under the
connection's
#  allowed_paths", "code": "path_not_allowed", ...}
```

The same goes for local files (`/etc/passwd`), other web addresses, `ATTACH`, `COPY ... TO`, installing or loading extensions, and attempts to change the settings back - each refused by DuckDB, which locks the connection's configuration as it opens.

Web files work the same way, with one difference: a web address in `allowed_paths` must name **a file** (`https://data.example.com/prices.parquet`), never a folder - a web server resolves `..` in a URL, so a prefix couldn't stop `https://data.example.com/public/../private/...`.

Good to know

- **Local files and folders** work the same way, with absolute paths: `"allowed_paths": ["/data/sales/"]`. A DuckDB connection with no `allowed_paths` reads no files at all.
- **Credentials are visible in part.** Anyone who can run SQL on the connection can see its key ID and endpoint through DuckDB's `duckdb_secrets()` - never the secret. Another reason to give partners saved queries, not SQL.
- **Costs**: on a cloud bucket, each read is requests and egress you pay for. Cache, rate-limit, and prefer Parquet - DuckDB reads only the columns and row groups a query needs.

Next steps

- Cache a saved query's response.
- Let an AI agent call your saved queries via MCP - the bucket, as tools.
- DuckDB connections reference.